

UNIVERSIDAD AUTÓNOMA DE BUCARAMANGA

Facultad de Ingeniería

Ingeniería de Sistemas

INFORME DE LABORATORIO 3

MQTT hacia Azure IoT Central: protocolo visible, mediciones y comparación con el SDK

Presentado por:

Juan Sebastian Lizcano Jaimes

Angie Katherine Suarez Ortiz

IoT + Cloud + Sistemas Distribuidos

Bucaramanga, septiembre de 2026

Resumen ejecutivo

El objetivo fue explicar qué protocolo usa realmente un dispositivo con Azure IoT Central, publicar las variables del Laboratorio 2 por el SDK Python y por un cliente MQTT explícito, mantener el ESP32 de Wokwi en la misma aplicación, medir payload, intervalo, tiempo de llegada y efecto de un corte, y documentar qué QoS acepta Central. Las capturas están en el Anexo A, en el orden del laboratorio.

1. Flujo MQTT hacia IoT Central

El dispositivo se registra en el DPS global (global.azure-devices-provisioning.net) por MQTT sobre TLS en el puerto 8883, se autentica con un token SAS y recibe el hub asignado. Luego abre una sesión MQTT 3.1.1 con ese hub, con username {hub}/{deviceId}/?api-version=2021-04-12, y publica en devices/{deviceId}/messages/events/. IoT Central lee los mensajes del hub y los interpreta con la plantilla Dispositivo Laboratorio 1. El puerto 443 es la alternativa con MQTT sobre WebSockets. El cliente explícito entra directo al hub ya asignado, sin pasar por DPS.

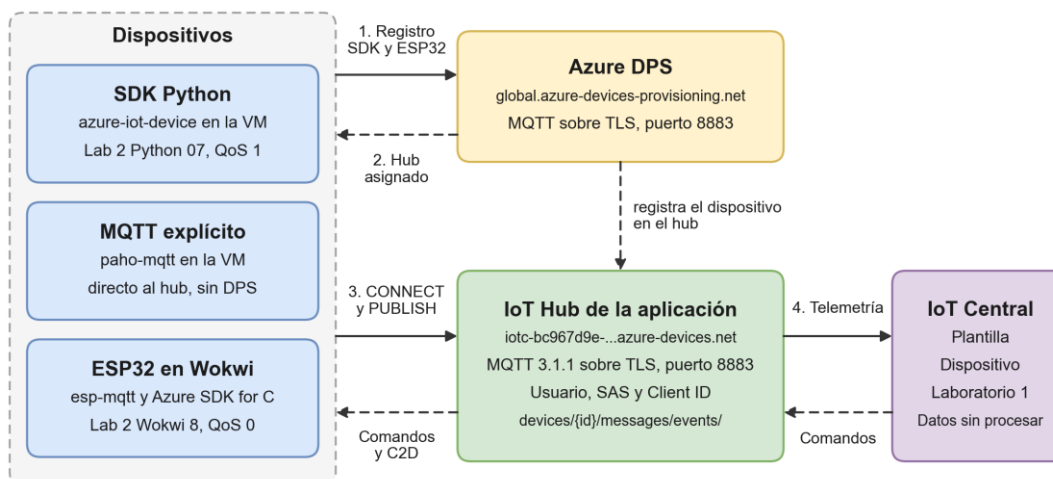


Figura 1. Flujo MQTT hacia DPS, IoT Hub e IoT Central en los tres caminos del laboratorio.

2. Procedimiento y puntos de control

Etapas 1. El DPS resolvió a 13.73.252.147 (South Central US), los puertos 8883 y 443 respondieron y openssl negoció TLS 1.3 con certificado válido. El SDK publica cada 5 s y el ESP32 cada 10 s de reloj simulado, ambos en el 8883, y el JSON coincide con la plantilla. Punto de control: el transporte es MQTT/TLS (Anexo A.2).

Etapas 2. mqtt_explicito.py genera la SAS con HMAC-SHA256, se conecta con paho-mqtt por TLS, publica las cuatro variables y se suscribe a C2D y a métodos directos. Usa el mismo lab2-python-07, nunca al tiempo con el SDK. Punto de control: el publish hecho a mano se ve en IoT Central (Anexo A.3).

Etapas 3. Series con QoS 0, 1 y 2, una serie de 10 s y un corte de 15 s; el CSV reunió 46 mensajes. Punto de control: la tabla de la sección 3 (Anexo A.4).

Etapas 4. El SDK del Laboratorio 2 y el ESP32 de Wokwi siguieron publicando en la misma aplicación (Anexo A.5). La comparación está en la sección 4.

3. Tabla de mediciones

Camino y serie	Payload JSON (bytes)	Intervalo configurado / observado	Tiempo del publish a Central	Corte de 15 s y restablecimiento	Resultado de QoS
MQTT explícito, QoS 0, N = 20	100 a 101	5 s / 5.00 s en el CSV; 5 s en Central	Menos de 1 s	No aplica	Aceptado, sin confirmación
MQTT explícito, QoS 1, N = 20	100 a 101	5 s / 5.13 a 5.15 s	Menos de 1 s; PUBACK de 136 a 144 ms	No aplica	Aceptado con PUBACK
MQTT explícito, QoS 2, N = 1	100	Un solo envío	Menos de 1 s; PUBACK de 143 ms	No aplica	Llega, confirmado como QoS 1
MQTT explícito, QoS 1, cada 10 s	100 a 101	10 s / 10.14 s; 10 s en Central	Menos de 1 s; PUBACK de 129 a 135 ms	No aplica	Aceptado con PUBACK
MQTT explícito, QoS 1, cada 2 s con corte	101	2 s / 2.13 s; 17.0 s en el corte	15.9 s el mensaje retenido	Sin reconexión ni pérdida; hueco de 18 s en Central	Mensaje pendiente entregado
SDK Python, 8 mensajes	109 a 110	5 s / 5 a 6 s en Central	Inmediato al refrescar	Por diseño: reconecta solo; QoS 1 conserva el mensaje	QoS 1 fijo del SDK
ESP32 Wokwi, 8 mensajes	Cerca de 100 (calculado)	10 s simulados / 11 a 18 s reales	Inmediato al refrescar	Por diseño: reconecta solo; QoS 0 sin reenvío	QoS 0 en telemetría

4. Comparación de los tres caminos

Criterio	SDK Python	MQTT explícito	Wokwi Arduino
Facilidad de implementación	Alta: resuelve DPS, SAS, reconexión y comandos.	Media: SAS, username, topics y callbacks a mano.	Baja: SDK de C, certificado raíz, NTP y PlatformIO.
Control del protocolo	Bajo: topic, QoS y token quedan ocultos.	Alto: QoS, topic, keepalive, SAS y PUBACK visibles.	Medio: el firmware se edita, pero el SDK de C arma topics y token.
Payload	109 a 110 bytes, JSON con espacios.	100 a 101 bytes, JSON compacto.	Cerca de 100 bytes, JSON compacto.
Latencia percibida	Inmediata al refrescar la vista.	Menos de 1 s; PUBACK de 129 a 145 ms.	Inmediata; el intervalo se alarga con la velocidad del simulador.
Reconexión	Automática dentro del SDK.	Aguantó un corte de 15 s; caídas largas exigen lógica propia.	Automática en esp-mqtt; lo enviado sin red con QoS 0 se pierde.
Utilidad para el escenario del Lab 1	Gateway o servicio que reporte varios paneles.	Diagnóstico, mediciones y equipos sin SDK de Azure.	Lo más cercano a un controlador real de panel con sensores.

Conclusión. Usaríamos el SDK cuando el equipo puede correr Python, C o C# y se busca una operación estable, porque ya resuelve aprovisionamiento, token, reconexión y comandos. Usaríamos un cliente MQTT directo para diagnosticar, medir o controlar el protocolo, o en equipos sin SDK de Azure, asumiendo la SAS, la reconexión y los topics de comandos.

5. Conclusión sobre QoS

MQTT define QoS 0 (a lo sumo una vez), QoS 1 (al menos una vez, con PUBACK) y QoS 2 (exactamente una vez, con PUBREC, PUBREL y PUBCOMP). IoT Central, vía IoT Hub, solo acepta 0 y 1: QoS 1 recibió PUBACK en 129 a 145 ms y conservó el mensaje en el corte, y QoS 2 llegó con un solo PUBACK, como QoS 1. Como no hay entrega exactamente una vez, recomendamos QoS 1 con número de secuencia para descartar duplicados, también en el ESP32, que hoy usa QoS 0.

Anexo A. Evidencias del procedimiento

Las capturas siguen el orden en que se hizo el laboratorio, desde la revisión de la conexión del Laboratorio 2 hasta la comparación con Wokwi.

A.1 Entorno de trabajo

Se usó la misma VM de Azure del Laboratorio 2, con Debian 12, Python 3.11 y el entorno virtual .venv. En ese entorno estaban azure-iot-device para el script del Laboratorio 2, y paho-mqtt, certifi y python-dotenv para el cliente explícito; pandas se usó para resumir el CSV. Las credenciales del dispositivo (hostname del hub, Device ID y clave) se cargan como variables de entorno desde un archivo .env que no se sube al repositorio. El ESP32 se ejecutó con el proyecto PlatformIO del Laboratorio 2 en Wokwi para VS Code. No se montó un broker local: todas las pruebas se hicieron contra IoT Central.

A.2 Etapa 1: apertura de la caja negra del Laboratorio 2

No se modificó el código del SDK. Desde la carpeta del proyecto Python en la VM se actualizaron los paquetes y se instalaron dnsutils y netcat-openbsd; ca-certificates y curl ya estaban en su última versión.

```
(.venv) lab2@Laboratorio-2:~/laboratorio2/python-vm$ sudo apt update
sudo apt install -y ca-certificates curl dnsutils netcat-openbsd
Get:1 file:/etc/apt/mirrors/debian.list Mirrorlist [37 B]
Get:2 file:/etc/apt/mirrors/debian-security.list Mirrorlist [46 B]
Hit:3 http://azure.deb.debian.cloud/debian bookworm InRelease
Get:4 http://azure.deb.debian.cloud/debian bookworm-updates InRelease [55.4 kB]
Get:5 http://azure.deb.debian.cloud/debian bookworm-backports InRelease [59.4 kB]
Hit:6 http://azure.deb.debian.cloud/debian-security bookworm-security InRelease
Fetched 115 kB in 1s (167 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20250419-deb12u1).
curl is already the newest version (7.88.1-10-deb12u5).
The following additional packages will be installed:
  bind9-dnsutils
The following NEW packages will be installed:
  bind9-dnsutils dnsutils netcat-openbsd
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 210 kB of archives.
After this operation, 646 kB of additional disk space will be used.
Get:1 file:/etc/apt/mirrors/debian-security.list Mirrorlist [46 B]
Get:2 file:/etc/apt/mirrors/debian.list Mirrorlist [37 B]
Get:3 http://azure.deb.debian.cloud/debian-security bookworm-security/main amd64 bind9-dnsutils amd64 1:9.18.49-1-deb12u2 [156 kB]
Get:4 http://azure.deb.debian.cloud/debian-security bookworm-security/main amd64 dnsutils all 1:9.18.49-1-deb12u2 [12.3
```

Figura A1. Actualización de paquetes e instalación de dnsutils y netcat-openbsd en la VM.

La resolución de global.azure-devices-provisioning.net pasó por Azure Traffic Manager y terminó en 13.73.252.147, una instancia de DPS en South Central US. Con netcat se comprobó que la VM llega a ese host por el puerto 8883, de MQTT sobre TLS, y por el 443, alternativa de MQTT sobre WebSockets.

```
(.venv) lab2@Laboratorio-2:~/laboratorio2/python-vm$ getent hosts global.azure-devices-provisioning.net
13.73.252.147 idsu-prod-sn-001-su-az-1p.southcentralus.cloudapp.azure.com global.azure-devices-provisioning.net id-prod-global-endpoint.trafficmanager.net
(.venv) lab2@Laboratorio-2:~/laboratorio2/python-vm$ nc -vz -w 10 global.azure-devices-provisioning.net 8883
Connection to global.azure-devices-provisioning.net (13.73.252.147) 8883 port [tcp/*] succeeded!
(.venv) lab2@Laboratorio-2:~/laboratorio2/python-vm$ nc -vz -w 10 global.azure-devices-provisioning.net 443
Connection to global.azure-devices-provisioning.net (13.73.252.147) 443 port [tcp/https] succeeded!
(.venv) lab2@Laboratorio-2:~/laboratorio2/python-vm$
```

Figura A2. Resolución DNS del DPS y prueba de los puertos 8883 y 443.

Con openssl s_client se hizo el saludo TLS contra el puerto 8883. El servidor negoció TLSv1.3 con el cifrado TLS_AES_256_GCM_SHA384 y la cadena de certificados se validó con código 0, así que por ese puerto viaja MQTT dentro de un canal TLS y no en texto plano.

```
(.venv) lab2@Laboratorio-2:~/Laboratorio2/python-vm$ openssl s_client
-connect global.azure-devices-provisioning.net:8883 \
-servername global.azure-devices-provisioning.net \
</dev/null 2>/dev/null |
grep -E "Protocol|Cipher|Verify return code"
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Verify return code: 0 (ok)
(.venv) lab2@Laboratorio-2:~/Laboratorio2/python-vm$
```

Figura A3. Negociación TLS 1.3 con el DPS en el puerto 8883.

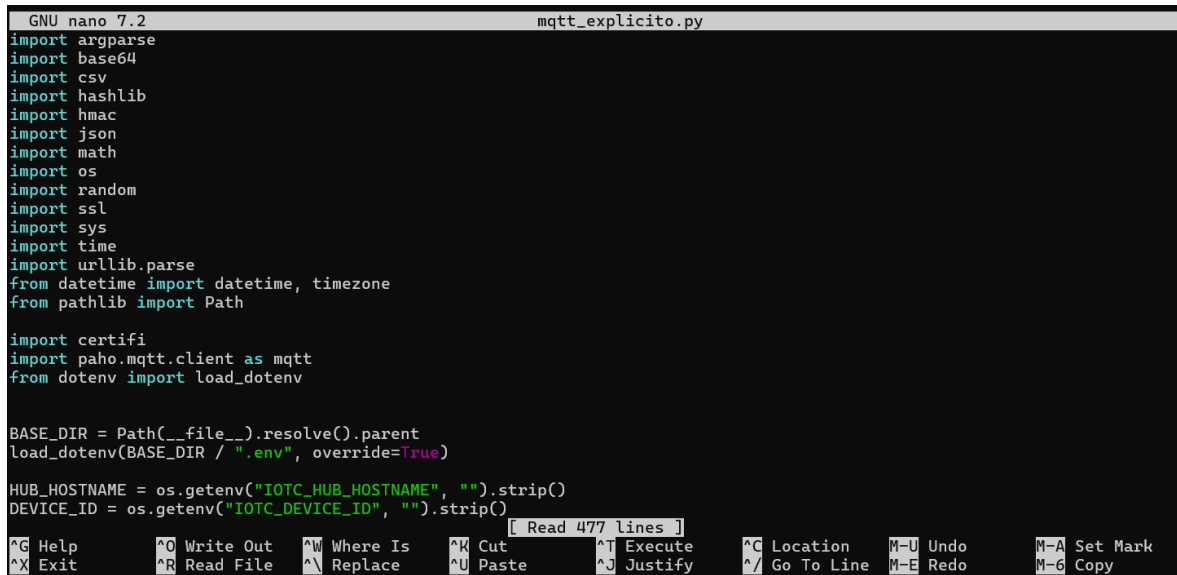
Con esta revisión, el registro del SDK (Figura A21) y el código del firmware del Laboratorio 2 se completó la siguiente tabla:

Dato	Lab 2 Python 07 (SDK)	Lab 2 Wokwi 8 (ESP32)
Primer salto	DPS en global.azure-devices-provisioning.net (13.73.252.147)	DPS en global.azure-devices-provisioning.net
Destino final	iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net	Hub asignado por DPS dentro de la misma aplicación
Puerto	8883	8883, con URI mqtt://
TLS	Sí, el endpoint negoció TLS 1.3 en la prueba con openssl	Sí, con certificado raíz incluido en el firmware
Autenticación	SAS generada por el SDK con la clave del dispositivo	SAS generada en el ESP32 con HMAC-SHA256
Publicación	Cada 5 s	Cada 10 s de reloj simulado
Variables	PotenciaGenerada, TemperaturaDelPanel, VoltajeDeSalida y CorrienteDeSalida	Las mismas cuatro variables

El JSON de telemetría se revisó en la pestaña Datos sin procesar del dispositivo, que muestra el mensaje tal como llega (Figura A14). Las claves PotenciaGenerada, TemperaturaDelPanel, VoltajeDeSalida y CorrienteDeSalida aparecen en columnas propias, lo que indica que IoT Central las reconoce como capacidades de la plantilla y no como datos sin modelar. Con esto se puede afirmar, sin adivinar, que el transporte hacia Central es MQTT sobre TLS.

A.3 Etapa 2: cliente MQTT explícito en la VM

En ~/laboratorio3/mqtt-explicito se creó mqtt_explicito.py. No usa azure-iot-device: toma el hostname del hub, el Device ID y la clave desde el archivo .env, genera la SAS con hmac, hashlib y base64, abre la conexión TLS con los certificados de certifi y maneja la sesión con paho-mqtt. Recibe por argumentos el QoS, la cantidad de mensajes y el intervalo, y guarda cada envío en un CSV.



```
GNU nano 7.2 mqtt_explicito.py
import argparse
import base64
import csv
import hashlib
import hmac
import json
import math
import os
import random
import ssl
import sys
import time
import urllib.parse
from datetime import datetime, timezone
from pathlib import Path

import certifi
import paho.mqtt.client as mqtt
from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parent
load_dotenv(BASE_DIR / ".env", override=True)

HUB_HOSTNAME = os.getenv("IOTC_HUB_HOSTNAME", "").strip()
DEVICE_ID = os.getenv("IOTC_DEVICE_ID", "").strip()

[ Read 477 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo      M-6 Copy
```

Figura A4. Encabezado de mqtt_explicito.py con las librerías para la SAS y el cliente paho-mqtt.

Para generar la SAS se toma como recurso {hub}/devices/lab2-python-07 codificado para URL, se firma con HMAC-SHA256 el texto formado por ese recurso y la fecha de expiración usando la clave del dispositivo decodificada en Base64, y el resultado se envía como contraseña de MQTT. La clave nunca se imprime en consola. Los parámetros de la sesión quedaron así:

Parámetro	Valor usado
Host y puerto	iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net, puerto 8883 con TLS
Versión de MQTT	3.1.1
Client ID	lab2-python-07
Username	{hub}/lab2-python-07/?api-version=2021-04-12
Password	SharedAccessSignature sr={recurso}&sig={firma}&se={expiración}
Topic de telemetría	devices/lab2-python-07/messages/events/
Suscripciones	devices/lab2-python-07/messages/devicebound/# para mensajes C2D y \$iothub/methods/POST/# para comandos
QoS	0, 1 o 2 según el argumento --qos

Se usó el mismo dispositivo del Laboratorio 2, lab2-python-07, para que los datos llegaran a la misma aplicación del Laboratorio 1 con la misma plantilla y los mismos nombres de telemetría. Como IoT Hub acepta una sola conexión activa por Device ID, el script con SDK y el cliente explícito nunca se ejecutaron al mismo tiempo. El cliente también queda suscrito a los topics de C2D

y de métodos directos, que es por donde IoT Central envía los comandos, así que el canal de bajada queda abierto en la misma sesión MQTT.

La primera ejecución fue una prueba de funcionamiento con QoS 1. Cada publicación muestra la secuencia, el topic, el QoS, el tamaño del payload, el intervalo observado y el tiempo hasta recibir el PUBACK, que estuvo entre 129 y 145 ms. Los mensajes publicados a mano aparecieron en Lab 2 Python 07 (Figuras A7, A10 y A14).

```
[PUBLISH] rc= 0 mid= 18
[PUBACK] mid=18, tiempo=144.95 ms

[PUBLISH] secuencia=15
[PUBLISH] topic= devices/lab2-python-07/messages/events/
[PUBLISH] qos= 1
[PUBLISH] payload_bytes= 101
[PUBLISH] intervalo_observado=5.146 s
[PUBLISH] payload= {"PotenciaGenerada":115.9,"TemperaturaDelPanel":38.2,"VoltajeDeSalida":20.2,"CorrienteDeSalida":5.74}
[PUBLISH] rc= 0 mid= 19
[PUBACK] mid=19, tiempo=129.92 ms

[PUBLISH] secuencia=16
[PUBLISH] topic= devices/lab2-python-07/messages/events/
[PUBLISH] qos= 1
[PUBLISH] payload_bytes= 101
[PUBLISH] intervalo_observado=5.130 s
[PUBLISH] payload= {"PotenciaGenerada":119.7,"TemperaturaDelPanel":39.2,"VoltajeDeSalida":20.6,"CorrienteDeSalida":5.81}
[PUBLISH] rc= 0 mid= 20
[PUBACK] mid=20, tiempo=138.89 ms

[PUBLISH] secuencia=17
[PUBLISH] topic= devices/lab2-python-07/messages/events/
[PUBLISH] qos= 1
[PUBLISH] payload_bytes= 101
[PUBLISH] intervalo_observado=5.139 s
[PUBLISH] payload= {"PotenciaGenerada":122.8,"TemperaturaDelPanel":38.9,"VoltajeDeSalida":20.7,"CorrienteDeSalida":5.93}
[PUBLISH] rc= 0 mid= 21
[PUBACK] mid=21, tiempo=129.21 ms
```

Figura A5. Primera ejecución del cliente explícito con QoS 1 y confirmaciones PUBACK.

A.4 Etapa 3: series de medición

Antes de las series se borró mediciones_mqtt.csv para dejar solo las pruebas formales. El CSV guarda secuencia, fecha UTC, QoS, tamaño del payload, intervalo configurado y topic; el intervalo observado y el tiempo de PUBACK quedan en evidencia-qos0.log, evidencia-qos1.log y evidencia-qos2.log, generados con tee.

A.4.1 Serie con QoS 0

Se enviaron 20 mensajes cada 5 s con QoS 0.

```
(.env) lab2@Laboratorio-2:~/laboratorio3/mqtt-explicito$ rm -f mediciones_mqtt.csv
(.env) lab2@Laboratorio-2:~/laboratorio3/mqtt-explicito$ python mqtt_explicito.py \
--qos 0 \
--count 20 \
--interval 5 |
tee evidencia-qos0.log
```

Figura A6. Serie de 20 mensajes con QoS 0 e intervalo de 5 s.

Los mensajes quedaron separados exactamente 5 s. El primer envío del CSV tiene hora 16:32:49.455 UTC, las 11:32:49 en Colombia, y aparece en Datos sin procesar en ese mismo segundo junto con el evento Dispositivo conectado, así que el tiempo del publish a Central fue menor a un segundo.

Marca de tiempo ↓	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia Generada	Temperatura del P
> 11/9/2026, 11:33:29	Telemetría		4.12	80.8	34.3
> 11/9/2026, 11:33:24	Telemetría		3.71	72	32.7
> 11/9/2026, 11:33:19	Telemetría		3.27	62.5	31.8
> 11/9/2026, 11:33:14	Telemetría		3.04	58.4	31.8
> 11/9/2026, 11:33:09	Telemetría		2.57	48.8	30.7
> 11/9/2026, 11:33:04	Telemetría		2.42	44.5	29.7
> 11/9/2026, 11:32:59	Telemetría		1.83	34	27.6
> 11/9/2026, 11:32:54	Telemetría		1.22	22.2	26.9

Figura A7. Telemetría de la serie QoS 0 en Datos sin procesar de Lab 2 Python 07.

En la misma vista quedaron los eventos de la prueba inicial: la última telemetría a las 11:31:25 y la desconexión a las 11:32:20. Central marcó la desconexión casi un minuto después del último mensaje, por lo que el estado de conexión no se actualiza al instante.

Marca de tiempo ↓	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia Generada	Temperatura del Panel	Voltaje de Salida
> 11/9/2026, 11:33:29	Telemetría		4.12	80.8	34.3	19.6
> 11/9/2026, 11:33:24	Telemetría		3.71	72	32.7	19.4
> 11/9/2026, 11:33:19	Telemetría		3.27	62.5	31.8	19.1
> 11/9/2026, 11:33:14	Telemetría		3.04	58.4	31.8	19.2
> 11/9/2026, 11:33:09	Telemetría		2.57	48.8	30.7	19
> 11/9/2026, 11:33:04	Telemetría		2.42	44.5	29.7	18.4
> 11/9/2026, 11:32:59	Telemetría		1.83	34	27.6	18.6
> 11/9/2026, 11:32:54	Telemetría		1.22	22.2	26.9	18.2
> 11/9/2026, 11:32:49	Telemetría		1.06	19	26.9	17.9
> 11/9/2026, 11:32:49	Dispositivo conectado					
> 11/9/2026, 11:32:20	Dispositivo desconectado					
> 11/9/2026, 11:31:25	Telemetría		6.11	124.6	39.2	20.4

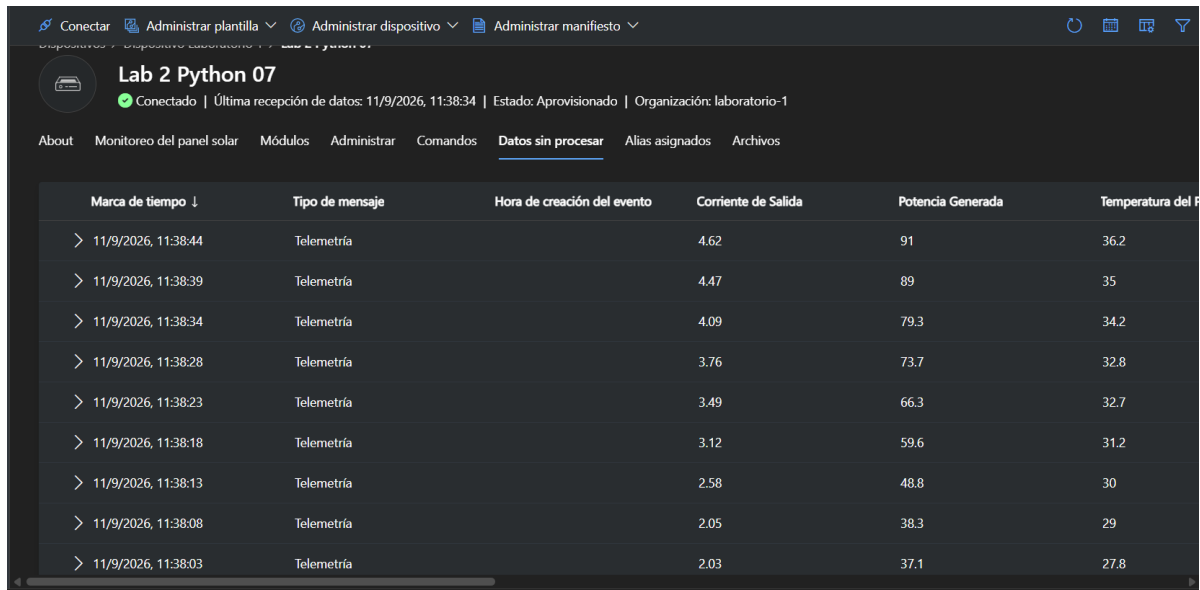
Figura A8. Eventos de conexión y desconexión alrededor de la serie QoS 0.

A.4.2 Serie con QoS 1

La misma serie se repitió con QoS 1.

```
(.venv) lab2@Laboratorio-2:~/laboratorio3/mqtt-explicito$ python mqtt_explicito.py \
--qos 1 \
--count 20 \
--interval 5 |
tee evidencia-qos1.log
```

Figura A9. Serie de 20 mensajes con QoS 1 e intervalo de 5 s.



The screenshot shows the IoT Central interface for a device named 'Lab 2 Python 07'. The device is connected, and the last data reception was on 11/9/2026 at 11:38:34. The status is 'Aprovisionado' and the organization is 'laboratorio-1'. The 'Datos sin procesar' (Raw Data) tab is selected, displaying a table of telemetry data.

Marca de tiempo ↓	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia Generada	Temperatura del F
> 11/9/2026, 11:38:44	Telemetría		4.62	91	36.2
> 11/9/2026, 11:38:39	Telemetría		4.47	89	35
> 11/9/2026, 11:38:34	Telemetría		4.09	79.3	34.2
> 11/9/2026, 11:38:28	Telemetría		3.76	73.7	32.8
> 11/9/2026, 11:38:23	Telemetría		3.49	66.3	32.7
> 11/9/2026, 11:38:18	Telemetría		3.12	59.6	31.2
> 11/9/2026, 11:38:13	Telemetría		2.58	48.8	30
> 11/9/2026, 11:38:08	Telemetría		2.05	38.3	29
> 11/9/2026, 11:38:03	Telemetría		2.03	37.1	27.8

Figura A10. Telemetría de la serie QoS 1 recibida en IoT Central.

Con QoS 1 el script espera el PUBACK de cada mensaje antes de contar el siguiente intervalo. Por eso el intervalo observado fue de 5.13 a 5.15 s: los 5 s configurados más los 130 a 145 ms de la confirmación. En Central se nota como un salto ocasional de 6 s entre marcas, por ejemplo de 11:38:28 a 11:38:34. Los 20 mensajes quedaron confirmados y la sesión cerró con desconexión normal.

```
[PUBLISH] secuencia=17
[PUBLISH] topic= devices/lab2-python-07/messages/events/
[PUBLISH] qos= 1
[PUBLISH] payload_bytes= 101
[PUBLISH] intervalo_observado=5.139 s
[PUBLISH] payload= {"PotenciaGenerada":119.5,"TemperaturaDelPanel":39.2,"VoltajeDeSalida":20.4,"CorrienteDeSalida":5.86}
[PUBLISH] rc= 0 mid= 21
[PUBACK] mid=21, tiempo=144.42 ms

[PUBLISH] secuencia=18
[PUBLISH] topic= devices/lab2-python-07/messages/events/
[PUBLISH] qos= 1
[PUBLISH] payload_bytes= 101
[PUBLISH] intervalo_observado=5.145 s
[PUBLISH] payload= {"PotenciaGenerada":123.0,"TemperaturaDelPanel":39.0,"VoltajeDeSalida":20.7,"CorrienteDeSalida":5.94}
[PUBLISH] rc= 0 mid= 22
[PUBACK] mid=22, tiempo=137.24 ms

[PUBLISH] secuencia=19
[PUBLISH] topic= devices/lab2-python-07/messages/events/
[PUBLISH] qos= 1
[PUBLISH] payload_bytes= 101
[PUBLISH] intervalo_observado=5.138 s
[PUBLISH] payload= {"PotenciaGenerada":122.5,"TemperaturaDelPanel":38.5,"VoltajeDeSalida":20.7,"CorrienteDeSalida":5.92}
[PUBLISH] rc= 0 mid= 23
[PUBACK] mid=23, tiempo=136.48 ms

[FIN] Mensajes enviados: 20
[DISCONNECT] Código: Normal disconnection
(.venv) lab2@Laboratorio-2:~/laboratorio3/mqtt-explicito$
```

Figura A11. Cierre de la serie QoS 1 con 20 mensajes enviados y desconexión normal.

A.4.3 Prueba con QoS 2

Para QoS 2 se envió un solo mensaje y se limitó la ejecución a 30 s con timeout, por si el servicio cerraba la conexión o el intercambio quedaba bloqueado.

```
(.venv) lab2@Laboratorio-2:~/laboratorio3/mqtt-explicito$ timeout 30 python mqtt_explicito.py \
--qos 2 \
--count 1 \
--interval 5 |
tee evidencia-qos2.log
```

Figura A12. Prueba de un mensaje con QoS 2 limitada a 30 s.

El encabezado del script resume la sesión: hub, puerto 8883, TLS activo, MQTT 3.1.1, Client ID, username con api-version=2021-04-12, topic de eventos y QoS solicitado 2. El mensaje quedó confirmado con un PUBACK a los 143.02 ms, igual que uno con QoS 1, y llegó a Central con las cuatro variables.

```

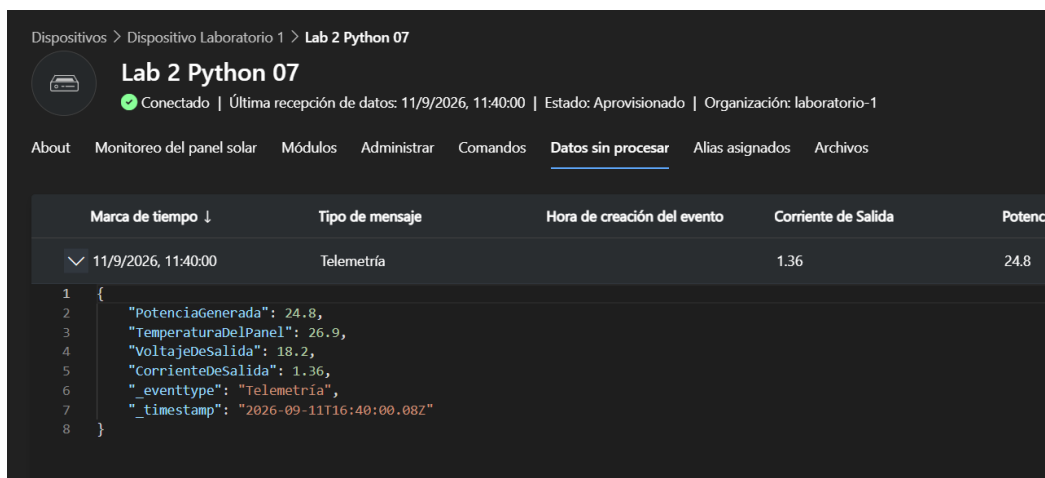
===== MQTT EXPLÍCITO =====
Hostname: iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net
Puerto: 8883
TLS: Si
Versión MQTT: 3.1.1
Client ID: lab2-python-07
Username: iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net/lab2-python-07/?api-version=2021-04-12
Topic: devices/lab2-python-07/messages/events/
QoS solicitado: 2
Clave: Cargada, no mostrada
=====
[CONNECT] Iniciando conexión...
[CONNECT] Código de respuesta: Success
[CONNECT] MQTT conectado.
[TLS] Puerto: 8883
[TLS] Cifrado activo: Si
[SUBSCRIBE] devices/lab2-python-07/messages/devicebound/#
[SUBSCRIBE] $iothub/methods/POST/#

[PUBLISH] secuencia=0
[PUBLISH] topic= devices/lab2-python-07/messages/events/
[PUBLISH] qos= 2
[PUBLISH] payload_bytes= 100
[PUBLISH] payload= {"PotenciaGenerada":24.8,"TemperaturaDelPanel":26.9,"VoltajeDeSalida":18.2,"CorrienteDeSalida":1.36}
[PUBLISH] rc= 0 mid= 4
[PUBACK] mid=4, tiempo=143.02 ms

[FIN] Mensajes enviados: 1
[DISCONNECT] Código: Normal disconnection
(.venv) lab2@Laboratorio-2:~/laboratorio3/mqtt-explicito$

```

Figura A13. Parámetros de conexión y resultado de la publicación con QoS 2.



Marca de tiempo ↓	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia
11/9/2026, 11:40:00	Telemetría		1.36	24.8

```

1 {
2   "PotenciaGenerada": 24.8,
3   "TemperaturaDelPanel": 26.9,
4   "VoltajeDeSalida": 18.2,
5   "CorrienteDeSalida": 1.36,
6   "_eventtype": "Telemetría",
7   "_timestamp": "2026-09-11T16:40:00.08Z"
8 }

```

Figura A14. JSON del mensaje con QoS 2 en Datos sin procesar.

El registro solo muestra un PUBACK; no aparece el intercambio PUBREC, PUBREL y PUBCOMP que define QoS 2 en la especificación MQTT. Como IoT Hub solo trabaja con QoS 0 y 1, en la práctica el mensaje se manejó como QoS 1. En el JSON también se ve que Central agrega los campos _eventtype y _timestamp.

A.4.4 Intervalo configurado y observado

Para comparar el intervalo configurado con el que muestra Central se hizo una serie corta con 10 s y QoS 1.

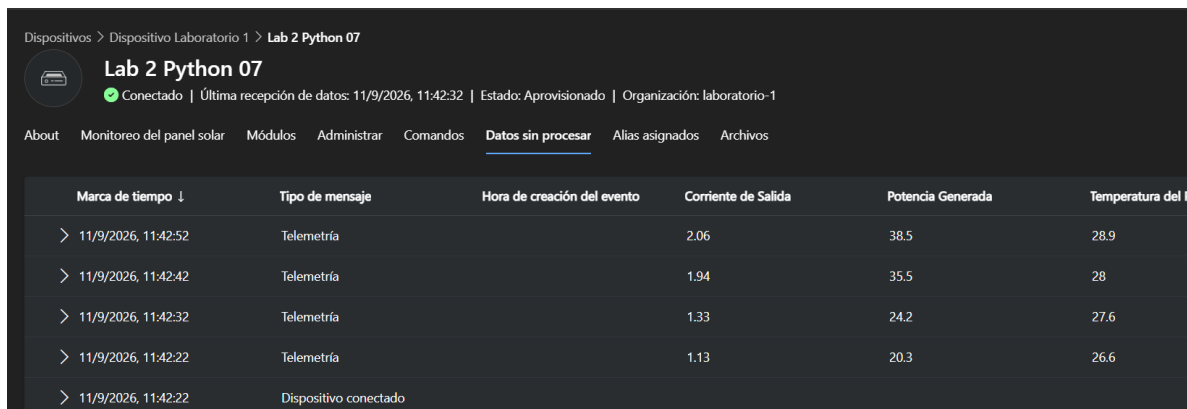
```
Username: iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net/lab2-python-07/?api-version=2021-04-12
Topic: devices/lab2-python-07/messages/events/
QoS solicitado: 1
Clave: Cargada, no mostrada
=====
[CONNECT] Iniciando conexión...
[CONNECT] Código de respuesta: Success
[CONNECT] MQTT conectado.
[TLS] Puerto: 8883
[TLS] Cifrado activo: Si
[SUBSCRIBE] devices/lab2-python-07/messages/devicebound/#
[SUBSCRIBE] $iothub/methods/POST/#

[PUBLISH] secuencia=0
[PUBLISH] topic= devices/lab2-python-07/messages/events/
[PUBLISH] qos= 1
[PUBLISH] payload_bytes= 100
[PUBLISH] payload= {"PotenciaGenerada":20.3,"TemperaturaDelPanel":26.6,"VoltajeDeSalida":18.0,"CorrienteDeSalida":1.13}
[PUBLISH] rc= 0 mid= 4
[PUBACK] mid=4, tiempo=134.89 ms

[PUBLISH] secuencia=1
[PUBLISH] topic= devices/lab2-python-07/messages/events/
[PUBLISH] qos= 1
[PUBLISH] payload_bytes= 100
[PUBLISH] intervalo_observado=10.136 s
[PUBLISH] payload= {"PotenciaGenerada":24.2,"TemperaturaDelPanel":27.6,"VoltajeDeSalida":18.2,"CorrienteDeSalida":1.33}
[PUBLISH] rc= 0 mid= 5
[PUBACK] mid=5, tiempo=128.75 ms
```

Figura A15. Serie con intervalo configurado de 10 s y QoS 1.

El script midió 10.136 s entre el primer y el segundo mensaje y Central mostró marcas cada 10 s: 11:42:22, 11:42:32, 11:42:42 y 11:42:52. Los 136 ms de diferencia son el tiempo del PUBACK, que no alcanza a verse porque Central muestra segundos enteros.



Marca de tiempo ↓	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia Generada	Temperatura del P
> 11/9/2026, 11:42:52	Telemetría		2.06	38.5	28.9
> 11/9/2026, 11:42:42	Telemetría		1.94	35.5	28
> 11/9/2026, 11:42:32	Telemetría		1.33	24.2	27.6
> 11/9/2026, 11:42:22	Telemetría		1.13	20.3	26.6
> 11/9/2026, 11:42:22	Dispositivo conectado				

Figura A16. Mensajes de la serie de 10 s en IoT Central.

A.4.5 Registro en CSV

El CSV permite revisar los envíos sin depender de capturas. En la serie con QoS 0 las marcas UTC están separadas 5.0 s porque el cliente no espera confirmación.

```
(.venv) lab2@Laboratorio-2:~/Laboratorio3/mqtt-explicito$ column -s, -t mediciones_mqtt.csv | head -20
secuencia  fecha_utc      qos  payload_bytes  intervalo_configurado_s  topic
0          2026-09-11T16:32:49.455547+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
1          2026-09-11T16:32:54.456302+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
2          2026-09-11T16:32:59.457026+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
3          2026-09-11T16:33:04.457723+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
4          2026-09-11T16:33:09.458376+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
5          2026-09-11T16:33:14.459045+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
6          2026-09-11T16:33:19.459781+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
7          2026-09-11T16:33:24.460504+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
8          2026-09-11T16:33:29.461171+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
9          2026-09-11T16:33:34.461862+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
10         2026-09-11T16:33:39.462551+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
11         2026-09-11T16:33:44.463191+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
12         2026-09-11T16:33:49.463820+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
13         2026-09-11T16:33:54.464500+00:00  0    101           5.0                    devices/lab2-python-07/messages/events/
14         2026-09-11T16:33:59.465165+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
15         2026-09-11T16:34:04.465797+00:00  0    101           5.0                    devices/lab2-python-07/messages/events/
16         2026-09-11T16:34:09.466434+00:00  0    100           5.0                    devices/lab2-python-07/messages/events/
17         2026-09-11T16:34:14.467110+00:00  0    101           5.0                    devices/lab2-python-07/messages/events/
18         2026-09-11T16:34:19.467770+00:00  0    101           5.0                    devices/lab2-python-07/messages/events/
(.venv) lab2@Laboratorio-2:~/Laboratorio3/mqtt-explicito$
```

Figura A17. Primeras filas de mediciones_mqtt.csv.

Con pandas se resumió el archivo: 46 mensajes entre las series de QoS 0, QoS 1, QoS 2 y la de 10 s, con payload mínimo de 100 bytes, máximo de 101 y promedio de 100.3. El byte de diferencia depende de cuántas cifras tenga cada valor; por ejemplo, PotenciaGenerada ocupa un carácter más cuando pasa de 100.

```
(.venv) lab2@Laboratorio-2:~/Laboratorio3/mqtt-explicito$ python - <<'PY'
import pandas as pd

df = pd.read_csv("mediciones_mqtt.csv")

print("Mensajes:", len(df))
print(
    "Payload mínimo:",
    df["payload_bytes"].min()
)
print(
    "Payload máximo:",
    df["payload_bytes"].max()
)
print(
    "Payload promedio:",
    round(df["payload_bytes"].mean(), 2)
)
print(
    "QoS probados:",
    sorted(df["qos"].unique())
)
PY
..
Mensajes: 46
Payload mínimo: 100
Payload máximo: 101
Payload promedio: 100.3
QoS probados: [np.int64(0), np.int64(1), np.int64(2)]
(.venv) lab2@Laboratorio-2:~/Laboratorio3/mqtt-explicito$
```

Figura A18. Resumen del CSV con pandas.

A.4.6 Corte de red de 15 s

Para simular un corte de Internet sin apagar la VM se bloqueó la salida al puerto 8883 durante 15 s con iptables; la regla se agrega, se espera 15 s y se borra en el mismo comando. En otra terminal, el cliente expícito publicaba con QoS 1 cada 2 s.

```
lab2@Laboratorio-2:~$ sudo ba [PUBLISH] payload_bytes= 101
iptables -I OUTPUT 1 \ [PUBLISH] intervalo_observado=2.135 s
-p tcp --dport 8883 \ [PUBLISH] payload= {"PotenciaGenerada":119.7,"TemperaturaDelPanel":38.7,"VoltajeDeSalida":20.6,"CorrienteDeSalida":5.81}
-j REJECT [PUBLISH] rc= 0 mid= 24
[PUBACK] mid=24, tiempo=15926.89 ms

sleep 15 [PUBLISH] secuencia=21
iptables -D OUTPUT \ [PUBLISH] topic= devices/lab2-python-07/messages/events/
-p tcp --dport 8883 \ [PUBLISH] qos= 1
-j REJECT [PUBLISH] payload_bytes= 101
[PUBLISH] intervalo_observado=17.002 s
lab2@Laboratorio-2:~$ sudo ba [PUBLISH] payload= {"PotenciaGenerada":118.7,"TemperaturaDelPanel":39.0,"VoltajeDeSalida":20.5,"CorrienteDeSalida":5.79}
iptables -I OUTPUT 1 \ [PUBLISH] rc= 0 mid= 25
-p tcp --dport 8883 \ [PUBACK] mid=25, tiempo=131.50 ms
-j REJECT [PUBLISH] secuencia=22
sleep 15 [PUBLISH] topic= devices/lab2-python-07/messages/events/
iptables -D OUTPUT \ [PUBLISH] qos= 1
-p tcp --dport 8883 \ [PUBLISH] payload_bytes= 101
-j REJECT [PUBLISH] intervalo_observado=2.132 s
lab2@Laboratorio-2:~$ sudo ba [PUBLISH] payload= {"PotenciaGenerada":117.1,"TemperaturaDelPanel":39.0,"VoltajeDeSalida":20.4,"CorrienteDeSalida":5.74}
iptables -I OUTPUT 1 \ [PUBLISH] rc= 0 mid= 26
-p tcp --dport 8883 \ [PUBACK] mid=26, tiempo=131.92 ms
-j REJECT [PUBLISH] secuencia=23
sleep 15 [PUBLISH] topic= devices/lab2-python-07/messages/events/
iptables -D OUTPUT \ [PUBLISH] qos= 1
-p tcp --dport 8883 \ [PUBLISH] payload_bytes= 101
-j REJECT [PUBLISH] intervalo_observado=2.132 s
lab2@Laboratorio-2:~$ [PUBLISH] payload= {"PotenciaGenerada":119.7,"TemperaturaDelPanel":38.4,"VoltajeDeSalida":20.5,"CorrienteDeSalida":5.84}
```

Figura A19. Bloqueo de 15 s del puerto 8883 y efecto en el cliente MQTT.

El mensaje publicado durante el bloqueo quedó pendiente y su PUBACK llegó a los 15926.89 ms, apenas se quitó la regla. El siguiente mensaje salió 17.002 s después del anterior y luego la serie volvió a 2.13 s. No aparece un nuevo CONNECT, así que la sesión MQTT se mantuvo.

En Central hay un hueco de 18 s entre 11:47:59 y 11:48:17. El mensaje retenido aparece a las 11:48:17 con los mismos valores del registro, 119.7 W y 38.7 °C: no hubo pérdida, solo retraso. La marca de tiempo corresponde a la llegada del mensaje y no a la creación del dato; la columna Hora de creación del evento está vacía porque ningún cliente envía ese valor.

Marca de tiempo ↓	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia Generada	Temperatura del P
> 11/9/2026, 11:48:27	Telemetría		5.36	107.7	37.5
> 11/9/2026, 11:48:25	Telemetría		5.67	116.2	38
> 11/9/2026, 11:48:23	Telemetría		5.84	119.7	38.4
> 11/9/2026, 11:48:20	Telemetría		5.74	117.1	39
> 11/9/2026, 11:48:18	Telemetría		5.79	118.7	39
> 11/9/2026, 11:48:17	Telemetría		5.81	119.7	38.7
> 11/9/2026, 11:47:59	Telemetría		5.96	122.8	38.6
> 11/9/2026, 11:47:57	Telemetría		5.97	121.8	39.2
> 11/9/2026, 11:47:55	Telemetría		5.8	117.7	38.5

Figura A20. Hueco de 18 s en IoT Central y recuperación de la serie después del corte.

A.5 Etapa 4: SDK y Wokwi en la misma aplicación

A.5.1 SDK Python del Laboratorio 2

Terminadas las pruebas del cliente explícito se volvió a ejecutar `panel_python_vm.py`, el script del Laboratorio 2 con `azure-iot-device`. El SDK hace el aprovisionamiento por DPS, recibe el hub asignado, se conecta y publica cada 5 s. En consola solo informa que la conexión y el envío fueron correctos; no muestra topic, QoS, token ni confirmaciones.

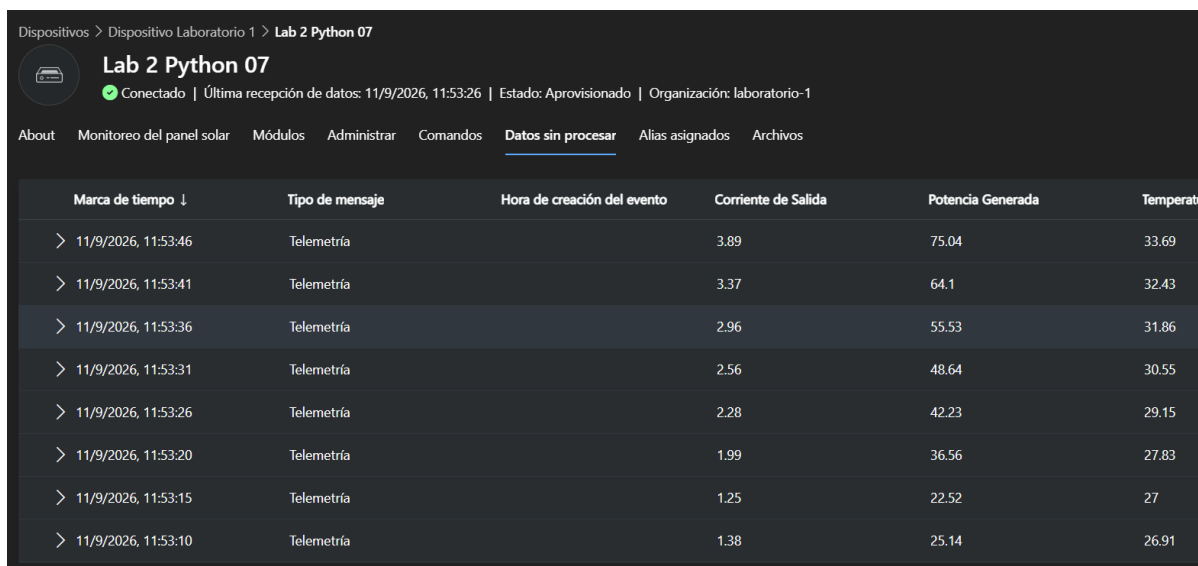
```
(.env) lab2@Laboratorio-2:~/laboratorio3/mqtt-explicito$ cd ~/Laboratorio2/python-vm
source .env/bin/activate
python panel_python_vm.py
Configuración cargada correctamente.
ID Scope: 0ne00FE096B
Device ID: lab2-python-07
Clave del dispositivo: cargada

Iniciando aprovisionamiento mediante Azure DPS...
Estado del aprovisionamiento: assigned
IoT Hub asignado: iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net
Device ID asignado: lab2-python-07

Conectando con Azure IoT Central...
Dispositivo conectado correctamente.
Telemetría enviada: {"PotenciaGenerada": 25.14, "TemperaturaDelPanel": 26.91, "VoltajeDeSalida": 18.22, "CorrienteDeSalida": 1.38}
Telemetría enviada: {"PotenciaGenerada": 22.52, "TemperaturaDelPanel": 27.0, "VoltajeDeSalida": 18.02, "CorrienteDeSalida": 1.25}
Telemetría enviada: {"PotenciaGenerada": 36.56, "TemperaturaDelPanel": 27.83, "VoltajeDeSalida": 18.37, "CorrienteDeSalida": 1.99}
Telemetría enviada: {"PotenciaGenerada": 42.23, "TemperaturaDelPanel": 29.15, "VoltajeDeSalida": 18.52, "CorrienteDeSalida": 2.28}
Telemetría enviada: {"PotenciaGenerada": 48.64, "TemperaturaDelPanel": 30.55, "VoltajeDeSalida": 19.0, "CorrienteDeSalida": 2.56}
Telemetría enviada: {"PotenciaGenerada": 55.53, "TemperaturaDelPanel": 31.86, "VoltajeDeSalida": 18.76, "CorrienteDeSalida": 2.96}
Telemetría enviada: {"PotenciaGenerada": 64.1, "TemperaturaDelPanel": 32.43, "VoltajeDeSalida": 19.02, "CorrienteDeSalida": 3.37}
```

Figura A21. Ejecución del script del Laboratorio 2 con aprovisionamiento DPS y envío de telemetría.

Los valores de la terminal coinciden con los de Datos sin procesar, por ejemplo 25.14 W y 26.91 °C a las 11:53:10. El script arma el JSON con espacios y dos decimales, por eso cada mensaje pesa entre 109 y 110 bytes según el texto que imprime la consola.



Marca de tiempo ↓	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia Generada	Temperatura
> 11/9/2026, 11:53:46	Telemetría		3.89	75.04	33.69
> 11/9/2026, 11:53:41	Telemetría		3.37	64.1	32.43
> 11/9/2026, 11:53:36	Telemetría		2.96	55.53	31.86
> 11/9/2026, 11:53:31	Telemetría		2.56	48.64	30.55
> 11/9/2026, 11:53:26	Telemetría		2.28	42.23	29.15
> 11/9/2026, 11:53:20	Telemetría		1.99	36.56	27.83
> 11/9/2026, 11:53:15	Telemetría		1.25	22.52	27
> 11/9/2026, 11:53:10	Telemetría		1.38	25.14	26.91

Figura A22. Telemetría del SDK Python en IoT Central.

Frente a un corte de red, el SDK tiene la reconexión automática activada y publica con QoS 1 sobre `paho-mqtt`, la misma librería del cliente explícito. Por eso un corte corto como el de la sección

A.4.6 debe traducirse en un envío retrasado y no en pérdida de datos; si la sesión llegara a caerse, el SDK la vuelve a abrir por su cuenta.

A.5.2 ESP32 en Wokwi

El proyecto del Laboratorio 2 se ejecutó sin cambios en Wokwi. El firmware usa el cliente esp-mqtt con URI `mqtt://` en el puerto 8883, genera la SAS dentro del ESP32 y publica la telemetría con QoS 0 cada 10 s.

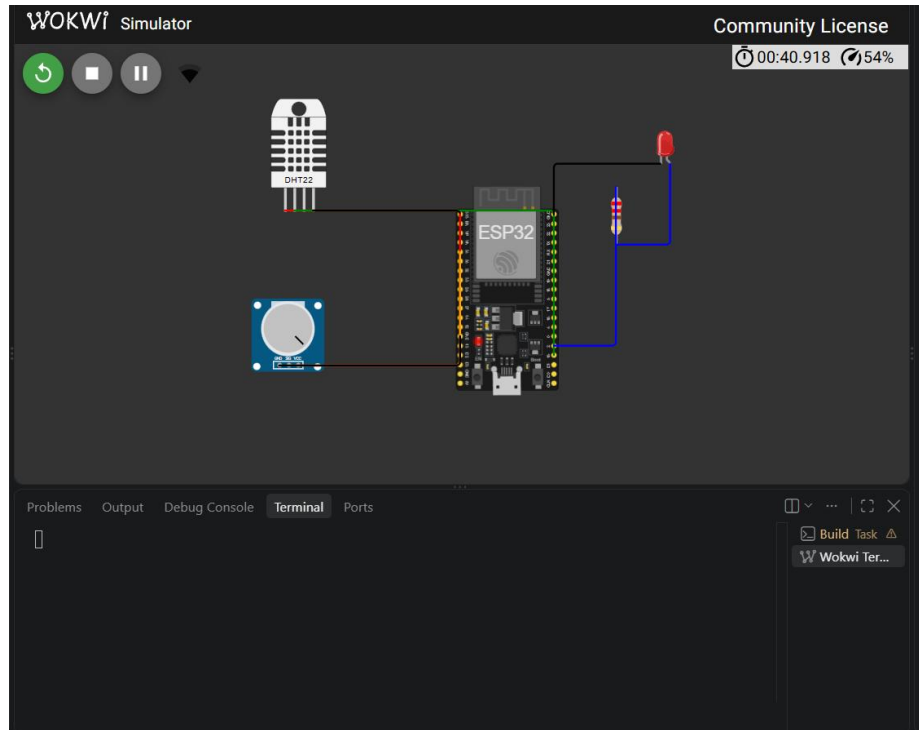


Figura A23. Circuito del ESP32 del Laboratorio 2 en ejecución en Wokwi.

Lab 2 Wokwi 8 apareció conectado en la misma aplicación. Primero reportó la propiedad con la información del dispositivo y después empezó a enviar telemetría. Las marcas quedaron separadas entre 11 y 17 s porque el simulador corría por debajo de la velocidad real, entre 54 y 75 % en las capturas, y los 10 s del firmware se cuentan con el reloj simulado.

Dispositivos > Dispositivo Laboratorio 1 > Lab 2 Wokwi 8					
Lab 2 Wokwi 8 Conectado Última recepción de datos: 11/9/2026, 11:57:09 Estado: Aprovisionado Organización: laboratorio-1					
About Monitoreo del panel solar Módulos Administrar Comandos <u>Datos sin procesar</u> Alias asignados Archivos					
Marca de tiempo ↓	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia Generada	Temperatura del F
> 11/9/2026, 11:57:09	Telemetría		8	192	23.8
> 11/9/2026, 11:56:52	Telemetría		8	192	23.8
> 11/9/2026, 11:56:38	Telemetría		3.27	53.9	23.8
> 11/9/2026, 11:56:27	Telemetría		3.27	53.9	23.8
ⓘ > 11/9/2026, 11:56:27	Propiedad				
> 11/9/2026, 11:56:26	Dispositivo conectado				

Figura A24. Lab 2 Wokwi 8 conectado y publicando en la misma aplicación.

Para confirmar que los valores salen del circuito se subió la temperatura del DHT22 a 58.8 °C y se movió el potenciómetro.

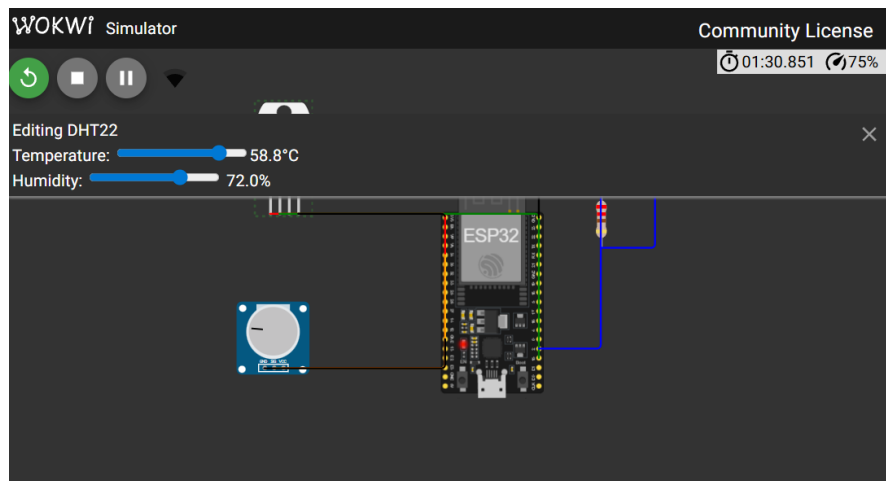


Figura A25. Cambio de temperatura del DHT22 y del potenciómetro en Wokwi.

IoT Central registró 58.7 °C en TemperaturaDelPanel, 1.88 A en CorrienteDeSalida y 26.8 W en PotenciaGenerada, con marcas cada 16 a 18 s. El ESP32 sigue publicando por MQTT en la misma aplicación que el SDK y el cliente explícito.

The image shows the IoT Central dashboard for a device named 'Lab 2 Wokwi 8'. The device is connected, and the last data reception was on 11/9/2026 at 11:58:53. The 'Datos sin procesar' (Raw Data) tab is selected, showing a table of telemetry data. The table has columns for 'Marca de tiempo' (Timestamp), 'Tipo de mensaje' (Message Type), 'Hora de creación del evento' (Event Creation Time), 'Corriente de Salida' (Output Current), 'Potencia Generada' (Generated Power), and 'Temperatura de' (Temperature of).

Marca de tiempo ↓	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia Generada	Temperatura de
> 11/9/2026, 11:58:53	Telemetría		1.88	26.8	58.7
> 11/9/2026, 11:58:35	Telemetría		1.88	26.8	58.7
> 11/9/2026, 11:58:18	Telemetría		1.88	26.8	58.7
> 11/9/2026, 11:58:02	Telemetría		1.88	26.8	58.7

Figura A26. Nuevos valores de Lab 2 Wokwi 8 en IoT Central después del cambio.

Ante un corte de red, el firmware reacciona así: si el Wi-Fi se cae, el loop detiene el cliente de Azure, espera a que el Wi-Fi vuelva y abre de nuevo la sesión con el hub sin repetir el aprovisionamiento por DPS, porque el hub asignado queda guardado en memoria. Además, esp-mqtt tiene la reconexión automática activada y un keepalive de 30 s. Como la telemetría sale con QoS 0 y no pasa por ninguna cola, las lecturas del tiempo sin conexión no se guardan ni se reenvían.

Repositorio GitHub

<https://github.com/Hypnostian/IOT.git>